



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 4: Unidade I —

Ponteiros para struct e vetores de struct

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade4.c`, evolução da `atividade3.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- Um **vetor de struct**: um único bloco contíguo de memória, alocado com `malloc`, contendo várias *structs* `Inimigo`;
- **Ponteiro para struct** recebido como parâmetro de função, permitindo alterar um elemento específico do vetor sem copiá-lo;
- **Ponteiro para struct** devolvido como retorno de função, permitindo localizar e apontar para um elemento específico do vetor;
- Um **enum** (`EstadoInimigo`) para representar o estado de cada elemento do vetor.

O programa exibe um jogador que atira no inimigo vivo mais próximo pressionando a barra de espaço.

1 Estrutura de dados: o inimigo

Cada inimigo é uma *struct* com posição, raio, vida e um campo do tipo **enum** indicando se ele está vivo ou morto:

```
1 typedef enum {  
2     INIMIGO_VIVO ,  
3     INIMIGO_MORTO  
4 } EstadoInimigo;  
5  
6 typedef struct {  
7     Vector2      pos;  
8     float        raio;  
9     int          vida;  
10    EstadoInimigo estado;  
11 } Inimigo;
```

2 O vetor de struct (bloco contíguo)

Diferentemente da atividade 5 (que ainda vamos ver), aqui os inimigos ficam em um **único bloco contíguo de memória**: uma só chamada de malloc reserva espaço para todos os TOTAL_INIMIGOS de uma vez.

```
1 Inimigo *inimigos = (Inimigo *)malloc(TOTAL_INIMIGOS * sizeof(Inimigo
   ));
2 inicializarInimigos(inimigos, TOTAL_INIMIGOS);
```

A função inicializarInimigos recebe esse vetor por **ponteiro para struct** (Inimigo *vetor) e usa aritmética de ponteiros (vetor + i) para preencher cada posição, exatamente como criarBolas fazia na atividade 1.

3 Ponteiro para struct como parâmetro: alterar um elemento

A função atingirInimigo recebe um **ponteiro para UM único elemento** do vetor e altera sua vida e estado diretamente na memória original:

```
1 void atingirInimigo(Inimigo *inimigo, int dano) {
2     if (inimigo == NULL || inimigo->estado == INIMIGO_MORTO) return;
3
4     inimigo->vida -= dano;
5     if (inimigo->vida <= 0) {
6         inimigo->vida = 0;
7         inimigo->estado = INIMIGO_MORTO;
8     }
9 }
```

4 Ponteiro para struct como retorno de função

A função encontrarInimigoMaisProximo percorre o vetor de struct e **devolve um ponteiro** para o elemento vivo mais próximo (ou NULL se não houver nenhum vivo). Note que nenhuma cópia de Inimigo é feita: apenas um endereço é retornado.

```
1 Inimigo *encontrarInimigoMaisProximo(Inimigo *vetor, int n, Vector2
   posJogador) {
2     Inimigo *maisProximo = NULL;
3     float menorDistancia = 0.0f;
4
5     for (int i = 0; i < n; i++) {
6         Inimigo *ini = (vetor + i);
7         if (ini->estado == INIMIGO_MORTO) continue;
8
9         float dx = ini->pos.x - posJogador.x;
10        float dy = ini->pos.y - posJogador.y;
11        float distancia = sqrtf(dx * dx + dy * dy);
12
13        if (maisProximo == NULL || distancia < menorDistancia) {
14            maisProximo = ini;
15            menorDistancia = distancia;
16        }
17    }
18    return maisProximo;
19 }
```

No laço principal, o ponteiro retornado é passado diretamente para atingirInimigo:

```

1 if (IsKeyPressed(KEY_SPACE)) {
2     Inimigo *alvo = encontrarInimigoMaisProximo(inimigos,
3         TOTAL_INIMIGOS, jogador);
4     atingirInimigo(alvo, DANO_TIRO);
5 }

```

Ciclo de vida da memória

Etapa	Vetor de struct (Inimigo *)
Alocação	<code>malloc(TOTAL_INIMIGOS * sizeof(Inimigo))</code>
Uso	<code>atingirInimigo() + encontrarInimigoMaisProximo()</code>
Liberação	<code>free(inimigos)</code>

5 Exercícios

Exercício 1 — Cura em área

Atualmente só existe uma função que causa dano a um inimigo por vez.

Tarefa: implemente `void curarTodos(Inimigo *vetor, int n, int cura)`, que percorre o vetor de struct e aumenta a vida de **todos** os inimigos vivos.

Dicas:

- Use um laço `for` com aritmética de ponteiros (`vetor + i`), igual às demais funções da atividade.
- Ignore inimigos com `estado == INIMIGO_MORTO`.
- Defina um teto de vida (por exemplo, 60) para a cura não ultrapassar o valor inicial.
- Associe a chamada dessa função a uma nova tecla, por exemplo `KEY_C`.

Exercício 2 — Encontrar o mais fraco

A função `encontrarInimigoMaisProximo` retorna um ponteiro para o inimigo vivo mais próximo do jogador.

Tarefa: implemente uma função análoga, `Inimigo *encontrarInimigoMaisFraco(Inimigo *vetor, int n)`, que retorna um **ponteiro** para o inimigo vivo com a **menor vida** atual.

Dicas:

- Percorra o vetor comparando `ini->vida` em vez da distância até o jogador.
- Ignore inimigos mortos e retorne `NULL` se todos estiverem mortos.
- Troque a tecla de ataque para usar essa nova função e observe como o comportamento do jogo muda.